

Compiling Typed APIs into Context-Free Languages

Breandan Considine, Clemente Pasti, Tim Vieira

August 13, 2026

Motivation: type-safe code completion

- ▶ Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- ▶ No ill-typed statements $\sigma \in \ell_\Gamma$ or well-typed statements $\sigma \notin \ell_\Gamma$
- ▶ For any $\sigma \in \Sigma^*$ such that $\exists \tau. \sigma\tau \in \ell_\Gamma$, show τ 's in shortlex order

Motivation: type-safe code completion

- ▶ Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- ▶ No ill-typed statements $\sigma \in \ell_\Gamma$ or well-typed statements $\sigma \notin \ell_\Gamma$
- ▶ For any $\sigma \in \Sigma^*$ such that $\exists \tau. \sigma\tau \in \ell_\Gamma$, show τ 's in shortlex order

```
1 #include <memory>
2 long* codes(); double* groups();
3 const char* text(); float* samples();
4 std::unique_ptr<int> fresh(), parse(const char*);
5 std::unique_ptr<int> average(float*), select(double&);
6 std::unique_ptr<int> combine(const char*, long*);
7 std::unique_ptr<int> schedule(const char*, long*, float*);
8
9 int main(int argc, char** args) {
10     std::unique_ptr<int> b[]
11 }
```

Motivation: type-safe code completion

- ▶ Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- ▶ No ill-typed statements $\sigma \in \ell_\Gamma$ or well-typed statements $\sigma \notin \ell_\Gamma$
- ▶ For any $\sigma \in \Sigma^*$ such that $\exists \tau. \sigma\tau \in \ell_\Gamma$, show τ 's in shortlex order

```
1 #include <memory>
2 long* codes(); double* groups();
3 const char* text(); float* samples();
4 std::unique_ptr<int> fresh(), parse(const char*);
5 std::unique_ptr<int> average(float*), select(double&);
6 std::unique_ptr<int> combine(const char*, long*);
7 std::unique_ptr<int> schedule(const char*, long*, float*);
8
9 int main(int argc, char** args) {
10     std::unique_ptr<int> b[5];
11 }
```

```
;
= fresh();
= average(samples());
= parse(text());
= select(groups()[argc]);
= combine(text(), codes());
= schedule(text(), codes(), samples());
```

Motivation: type-safe code completion

- ▶ Construct a $\ell_\Gamma = \mathcal{L}(G_\Gamma)$ consisting of well-typed statements
- ▶ No ill-typed statements $\sigma \in \ell_\Gamma$ or well-typed statements $\sigma \notin \ell_\Gamma$
- ▶ For any $\sigma \in \Sigma^*$ such that $\exists \tau. \sigma\tau \in \ell_\Gamma$, show τ 's in shortlex order

```
1 #include <memory>
2 long* codes(); double* groups();
3 const char* text(); float* samples();
4 std::unique_ptr<int> fresh(), parse(const char*);
5 std::unique_ptr<int> average(float*), select(double&);
6 std::unique_ptr<int> combine(const char*, long*);
7 std::unique_ptr<int> schedule(const char*, long*, float*);
8
9 int main(int argc, char** args) {
10     std::unique_ptr<int> b[4];
11 }
```

? Why shortlex order?

- Parsimonious abduction
- Bounded-delay enumeration
- Efficient rank and unrank queries

```
;
= fresh();
= average(samples());
= parse(text());
= select(groups()[argc]);
= combine(text(), codes());
= schedule(text(), codes(), samples());
```

Prior work on type-safe code completion

Legend ✓ by construction **gen** generation-time **filt** post-generation ~ partial / empirical — absent / unclaimed

Category

Literature

Language

Output unit

Query format

Core theory

Ranking

Learning

Filter timing

Syntax safe

Type safe

Statements

Parametricity

Usable tool

Public URL

Prior work on type-safe code completion

Legend ✓ by construction **gen** generation-time **filt** post-generation  partial / empirical — absent / unclaimed

Category	Type-directed search		
Literature	Perelman'12	Gvero'13	Pelsmaecker'22
Language	C#	Scala	Statix DSL
Output unit	expression	expression	fragment
Query format	partial expr	typed hole	cursor
Core theory	bespoke	STLC	SDF3/Statix
Ranking	type distance	usage priors	solver order
Learning	—		—
Filter timing	search	search	solver
Syntax safe	✓	✓	✓
Type safe	✓	✓	✓
Statements	—	—	
Parametricity	—	—	✓
Usable tool	VS Code	Eclipse	Spoofax
Public URL	—	code	artifact

Prior work on type-safe code completion

Legend  by construction  generation-time  post-generation  partial / empirical  absent / unclaimed

Category	Type-directed search			Learned completion	
Literature	Perelman'12	Gvero'13	Pelsmaecker'22	Raychev'14	Semenkin'24
Language	C#	Scala	Statix DSL	Java	Py+JVM+JS
Output unit	expression	expression	fragment	call sequence	full line
Query format	partial expr	typed hole	cursor	holes	left prefix
Core theory	bespoke	STLC	SDF3/Statix	bespoke	—
Ranking	type distance	usage priors	solver order	RNN	LLM
Learning	—		—		
Filter timing	search	search	solver	rank only	
Syntax safe					
Type safe					—
Statements	—	—			
Parametricity	—	—		—	—
Usable tool	VS Code	Eclipse	Spoofax	Slang	JetBrains
Public URL	—	code	artifact	—	plugin

Prior work on type-safe code completion

Legend  by construction  generation-time  post-generation  partial / empirical  absent / unclaimed

Category	Type-directed search			Learned completion		Constrained
Literature	Perelman'12	Gvero'13	Pelsmaecker'22	Raychev'14	Semenkin'24	Mündler'25
Language	C#	Scala	Statix DSL	Java	Py+JVM+JS	TypeScript
Output unit	expression	expression	fragment	call sequence	full line	function
Query format	partial expr	typed hole	cursor	holes	left prefix	prompt prefix
Core theory	bespoke	STLC	SDF3/Statix	bespoke	—	prefix FSA
Ranking	type distance	usage priors	solver order	RNN	LLM	LLM
Learning	—		—			
Filter timing	search	search	solver	rank only		
Syntax safe						
Type safe					—	
Statements	—	—				
Parametricity	—	—		—	—	
Usable tool	VS Code	Eclipse	Spoofax	Slang	JetBrains	Git repo
Public URL	—	code	artifact	—	plugin	code

Prior work on type-safe code completion

Legend  by construction  generation-time  post-generation  partial / empirical  absent / unclaimed

Category	Type-directed search			Learned completion		Constrained
Literature	Perelman'12	Gvero'13	Pelsmaecker'22	Raychev'14	Semenkin'24	Mündler'25
Language	C#	Scala	Statix DSL	Java	Py+JVM+JS	TypeScript
Output unit	expression	expression	fragment	call sequence	full line	function
Query format	partial expr	typed hole	cursor	holes	left prefix	prompt prefix
Core theory	bespoke	STLC	SDF3/Statix	bespoke	—	prefix FSA
Ranking	type distance	usage priors	solver order	RNN	LLM	LLM
Learning	—		—			
Filter timing	search	search	solver	rank only		
Syntax safe						
Type safe					—	
Statements	—	—				
Parametricity	—	—		—	—	
Usable tool	VS Code	Eclipse	Spoofax	Slang	JetBrains	Git repo
Public URL	—	code	artifact	—	plugin	code

Gap Prior work tends to trade off probabilistic ranking, completion length, and type safety but does not guarantee bounded-delay enumeration for all and only well-typed statements.

How long are typical source code statements?

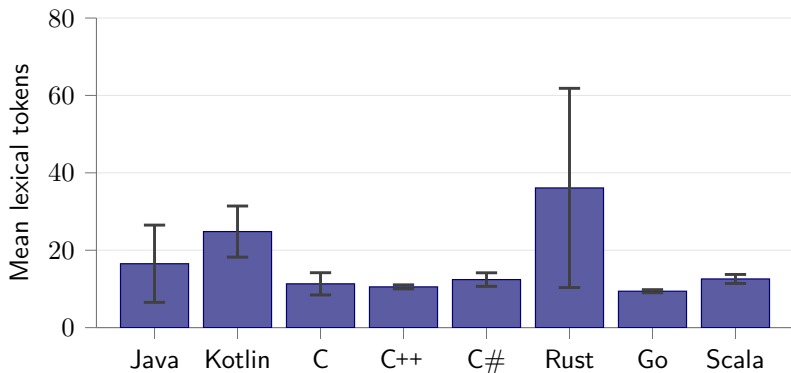


Figure: Whiskers are 95% CIs over sampled files. Statements are Tree-sitter statement nodes with comments and whitespace ignored. Samples are random GitHub files > 20kb; per-language sample sizes are shown below.

How large are programming language grammars?

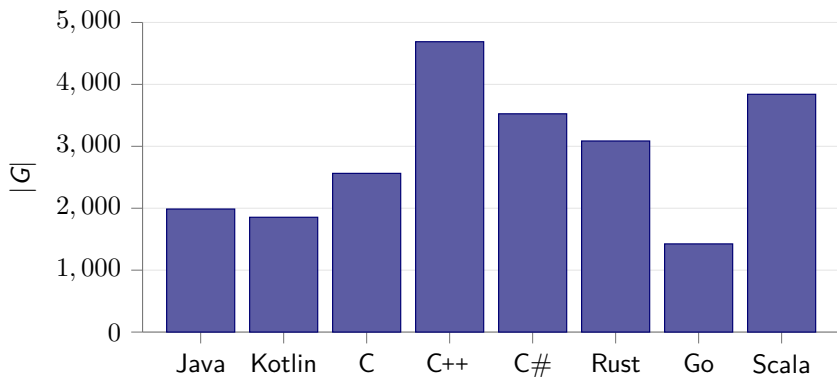


Figure: C++ is a syntactically feature-rich language. Grammar size calculated from Tree-sitter reference CFGs, where $|G| := \sum_{\substack{w \in V \\ (w \rightarrow \sigma) \in P}} |w\sigma|$.

Correctness criterion: what must the CFG preserve?

Surface fragment G'

e.g., names, invocation, member chains

$S ::= E; \quad E ::= \text{ID} \mid \text{ID}() \mid \text{ID}(\text{ARGS}) \mid E.E \quad \text{ARGS} ::= E \mid E, \text{ARGS}$

Correctness criterion: what must the CFG preserve?

Surface fragment G'

e.g., names, invocation, member chains

$S ::= E; \quad E ::= \text{ID} \mid \text{ID}() \mid \text{ID}(\text{ARGS}) \mid E.E \quad \text{ARGS} ::= E \mid E, \text{ARGS}$

1. Surface projection π_Σ

erase names but preserve punctuation

$$\pi_\Sigma(\mathbf{x.f(y)}) = \mathbf{ID.ID(ID)}$$

checks that the shape belongs to G_0

Correctness criterion: what must the CFG preserve?

Surface fragment G'

e.g., names, invocation, member chains

$S ::= E; \quad E ::= \text{ID} \mid \text{ID}() \mid \text{ID}(\text{ARGS}) \mid E.E \quad \text{ARGS} ::= E \mid E, \text{ARGS}$

1. Surface projection π_Σ

erase names but preserve punctuation

$\pi_\Sigma(\mathbf{x.f(y)}) = \text{ID.ID(ID)}$

checks that the shape belongs to G_0

2. Typed projection Π_Γ

names $\rightarrow$ type slots; API names stay

$\Pi_\Gamma(\mathbf{x.f(y)}) \ni E\langle X \rangle.f(E\langle Y \rangle)$

contains all and only well-typed terms

Correctness criterion: what must the CFG preserve?

Surface fragment G'

e.g., names, invocation, member chains

$S ::= E; \quad E ::= \text{ID} \mid \text{ID}() \mid \text{ID}(\text{ARGS}) \mid E.E \quad \text{ARGS} ::= E \mid E, \text{ARGS}$

1. Surface projection π_Σ

erase names but preserve punctuation

$$\pi_\Sigma(\mathbf{x.f(y)}) = \text{ID.ID(ID)}$$

checks that the shape belongs to G_0

2. Typed projection Π_Γ

names $\rightarrow$ type slots; API names stay

$$\Pi_\Gamma(\mathbf{x.f(y)}) \ni E\langle X \rangle.f(E\langle Y \rangle)$$

contains all and only well-typed terms

Correctness criterion

For a target language L with surface fragment G' , construct G_Γ satisfying

$$\pi_\Sigma(\sigma) \in \mathcal{L}(G') \iff \Gamma \vdash \Pi_\Gamma(\sigma) \in \mathcal{L}(G_\Gamma)$$

for all and only source statements σ accepted by a reference compiler for L .

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$G_{C++} \cup$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$G_{C++} \cup$

$G_{\text{<string>}} \cup$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$G_{\text{C++}} \cup$

$G_{\text{<string>}} \cup$

$G_{\text{<vector>}} \cup$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$G_{\text{C++}} \cup$$
$$G_{\text{<string>}} \cup$$
$$G_{\text{<vector>}} \cup$$
$$G_{\text{org/fizz.hpp}} \cup$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$G_{C++} \cup$$
$$G_{\langle \text{string} \rangle} \cup$$
$$G_{\langle \text{vector} \rangle} \cup$$
$$G_{\text{org/fizz.hpp}} \cup$$
$$G_{\text{com/buzz.hpp}} \cup$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$\begin{aligned} &G_{\text{C++}} \cup \\ &G_{\text{<string>}} \cup \\ &G_{\text{<vector>}} \cup \\ &G_{\text{org/fizz.hpp}} \cup \\ &G_{\text{com/buzz.hpp}} \cup \\ &\{ \text{Type} \rightarrow \text{string} \} \cup \end{aligned}$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$G_{\text{C++}} \cup$
 $G_{\text{<string>}} \cup$
 $G_{\text{<vector>}} \cup$
 $G_{\text{org/fizz.hpp}} \cup$
 $G_{\text{com/buzz.hpp}} \cup$
 $\{ \text{Type} \rightarrow \text{string} \} \cup$
...

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$\begin{aligned} &G_{\text{C++}} \cup \\ &G_{\text{<string>}} \cup \\ &G_{\text{<vector>}} \cup \\ &G_{\text{org/fizz.hpp}} \cup \\ &G_{\text{com/buzz.hpp}} \cup \\ &\{ \text{Type} \rightarrow \text{string} \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{fizz()}, \text{S} \rightarrow \text{fizz();} \} \cup \end{aligned}$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$\begin{aligned} &G_{\text{C++}} \cup \\ &G_{\text{<string>}} \cup \\ &G_{\text{<vector>}} \cup \\ &G_{\text{org/fizz.hpp}} \cup \\ &G_{\text{com/buzz.hpp}} \cup \\ &\{ \text{Type} \rightarrow \text{string} \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{fizz()}, S \rightarrow \text{fizz}(); \} \cup \\ &\{ \text{Str} \rightarrow \text{buzz(Str)}, S \rightarrow \text{buzz(Str)}; \} \cup \end{aligned}$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$\begin{aligned} &G_{\text{C++}} \cup \\ &G_{\text{<string>}} \cup \\ &G_{\text{<vector>}} \cup \\ &G_{\text{org/fizz.hpp}} \cup \\ &G_{\text{com/buzz.hpp}} \cup \\ &\{ \text{Type} \rightarrow \text{string} \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{fizz()}, S \rightarrow \text{fizz}(); \} \cup \\ &\{ \text{Str} \rightarrow \text{buzz(Str)}, S \rightarrow \text{buzz(Str)}; \} \cup \\ &\dots \end{aligned}$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$G_{\text{C++}} \cup$
 $G_{\text{<string>}} \cup$
 $G_{\text{<vector>}} \cup$
 $G_{\text{org/fizz.hpp}} \cup$
 $G_{\text{com/buzz.hpp}} \cup$
 $\{ \text{Type} \rightarrow \text{string} \} \cup$
 $\dots$
 $\{ \text{Int} \rightarrow \text{fizz()}, S \rightarrow \text{fizz()}; \} \cup$
 $\{ \text{Str} \rightarrow \text{buzz(Str)}, S \rightarrow \text{buzz(Str)}; \} \cup$
 $\dots$
 $\{ \text{Int} \rightarrow \text{argc}, \text{CharPtr} \rightarrow \text{args[Int]} \} \cup$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$\begin{aligned} &G_{\text{C++}} \cup \\ &G_{\text{<string>}} \cup \\ &G_{\text{<vector>}} \cup \\ &G_{\text{org/fizz.hpp}} \cup \\ &G_{\text{com/buzz.hpp}} \cup \\ &\{ \text{Type} \rightarrow \text{string} \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{fizz()}, S \rightarrow \text{fizz()}; \} \cup \\ &\{ \text{Str} \rightarrow \text{buzz(Str)}, S \rightarrow \text{buzz(Str)}; \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{argc}, \text{CharPtr} \rightarrow \text{args[Int]} \} \cup \\ &\{ \text{Str} \rightarrow \text{bang} \} \rightarrow G_{\Gamma} \end{aligned}$$

Grammar modularization

C++ context

```
1 // Filetype: *.cpp
2 #include <string>
3 #include <vector>
4 #include "org/fizz.hpp"
5 #include "com/buzz.hpp"
6 using std::string;
7 // ...
8 int fizz() { return 3; }
9 string buzz(string s) { return s; }
10 // ...
11 int main(int argc, char* args[]) {
12     string bang = "...";
13     □
14 }
```

Grammar contribution

$$\begin{aligned} &G_{\text{C++}} \cup \\ &G_{\text{<string>}} \cup \\ &G_{\text{<vector>}} \cup \\ &G_{\text{org/fizz.hpp}} \cup \\ &G_{\text{com/buzz.hpp}} \cup \\ &\{ \text{Type} \rightarrow \text{string} \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{fizz()}, S \rightarrow \text{fizz}(); \} \cup \\ &\{ \text{Str} \rightarrow \text{buzz}(\text{Str}), S \rightarrow \text{buzz}(\text{Str}); \} \cup \\ &\dots \\ &\{ \text{Int} \rightarrow \text{argc}, \text{CharPtr} \rightarrow \text{args}[\text{Int}] \} \cup \\ &\{ \text{Str} \rightarrow \text{bang} \} \rightarrow G_{\Gamma} \\ &\Gamma \vdash \mathcal{L}(G_{\Gamma}) \end{aligned}$$

Nominal typing judgements

Context

Γ contains all declarations visible at the cursor

$\Gamma \vdash e : T$

C++ accepts e and assigns the concrete type T .

C++ program

```
struct Tag {  
    int read(int n) {  
        return n + 1;  
    }  
};  
int main() {  
    Tag m;  
    int n = 2;  
    int x = m.read(n);  
    return x + x;  
}
```

Embedding

$\Gamma \vdash m.read(n) : int$

$\Downarrow$

$E\langle int \rangle \rightarrow E\langle Tag \rangle.read(E\langle int \rangle)$

$\Gamma \vdash x + x : int$

$\Downarrow$

$E\langle int \rangle \rightarrow E\langle int \rangle + E\langle int \rangle$

Embedding Substitute locally bound names with typed slots.

Subtyping rules

Derived-to-base call

`Dog <: Animal`

`Dog` publicly inherits `Animal`, so `d` may be passed to `age` as an `Animal`.

C++ program

```
struct Animal {};  
struct Dog : Animal {};  
int age(Animal) { return 7; }  
int main() {  
    Dog d;  
    int x = age(d);  
    return x;  
}
```

Embedding

$\Gamma \vdash \text{age} : \text{Animal} \rightarrow \text{int}$

$\Downarrow$

$E\langle \text{int} \rangle \rightarrow \text{age}(E\langle \text{Dog} \rangle)$

$E\langle \text{int} \rangle \rightarrow \text{age}(E\langle \text{Animal} \rangle)$

Scoped subtype inclusion

Only track well-scoped subtypes and embed each subtype monomorphically.

Overload resolution

Ambiguous candidates

```
struct A {}; struct B {};  
struct D : A, B {};  
int f(A); int f(B);  
D d; A a; B b;  
int n = f(a);  
int m = f(b);  
int o = f(d); // ambiguous
```

C++ resolution fails

$f(A) : D \rightarrow A$ $f(B) : D \rightarrow B$

Both candidates are viable by conversion but unresolvable.

$\text{resolver}_T(f(d)) = ?$

Caveat: local subtype closure is unsound

$$\left. \begin{array}{l} E\langle \text{int} \rangle \rightarrow f(E\langle X \rangle) \\ E\langle X \rangle \rightarrow E\langle D \rangle \end{array} \right\} X \in \{A, B\} \Rightarrow_{G_T} E\langle \text{int} \rangle \Rightarrow^* f(E\langle D \rangle)$$

Local subtype closure derives $f(E\langle D \rangle)$, but C++ rejects $f(d)$ as ambiguous.

Resolve before lowering. Emit a ground call production only when the exact call has a unique best viable function; ambiguous calls yield no productions.

Covariance and contravariance

Subtype setup

```
struct A {}; struct A_: A {};  
struct B {}; struct B_: B {};  
B f(A); B_ g(A_);  
A a; A_ a_;
```

Input position: contravariant

✓ `f(a_); f(a);`

✗ `g(a);` **Error**

An `A_` value may be passed to a `A` parameter but not vice versa.

Result position: covariant

✓ `B b = g(a_);`

✗ `B_ b_ = f(a);` **Error**

A `B_` result may widen to `B`; a `B` result cannot narrow to `B_`.

Embedding

$I\langle T \rangle$: admissible as an initializer of T

$E\langle B \rangle \rightarrow f(E\langle A \rangle) \mid f(E\langle A_{<:} \rangle) \quad E\langle B_{<:} \rangle \rightarrow g(E\langle A_{<:} \rangle) \quad I\langle B \rangle \rightarrow E\langle B \rangle \mid E\langle B_{<:} \rangle$

Template handling

C++ program

```
#include <string>
#include <vector>
using std::string;
template<class T>
T twice(T x) { return x+x; }
int main() {
    string s;
    std::vector<int> d;
    twice(1); // ok
    twice(s); // ok
    twice(d); // + undefined
}
```

Try concrete types

int	✓ accepted
std::string	✓ accepted
std::vector<int>	✗ rejected

The vector case fails because $x+x$ is not defined for vectors.

Filter successful rules

$$\Gamma \vdash \text{twice}(s) : \text{std::string} \Rightarrow E(\text{std::string}) \rightarrow \text{twice}(E(\text{std::string}))$$

Sound: every kept rule compiles **Complete (in scope):** all scoped types were tried

Template handling: invariant by default

Subtyping under parametricity

$\text{Dog} <: \text{Animal} \not\Rightarrow F\langle\text{Dog}\rangle <: F\langle\text{Animal}\rangle$

Class templates may optionally provide a constrained converting constructor.

API-provided covariance

```
std::shared_ptr<Dog> pd;  
std::shared_ptr<Animal> pa = pd;  
// accepted
```

`shared_ptr` explicitly supplies the conversion.

Invariant specialization

```
std::vector<Dog> ds;  
std::vector<Animal> as = ds;  
// rejected
```

The two `vector` specializations are unrelated.

CFG generation

Cannot infer that $F\langle D \rangle$ converts to $F\langle B \rangle$ from $D <: B$ alone. Emit a rule only when C++ library supplies and accepts a specific conversion.

Experimental Design

Dataset

- ▶ Dataset of standalone .cpp files; each compiles cleanly
- ▶ Hold out one whole statement at its original location
- ▶ Build G_Γ from context Γ
- ▶ Try to reconstruct from LTR tokenwise unmasking

Whole-statement measures

Precision reference-compiler acceptance rate of token-bounded uniform samples from $\mathcal{L}(G_\Gamma)$.

Recall fraction of whole statements admitted after fresh-name/literal normalization.

Name and literal normalization

Held-out statement x^*

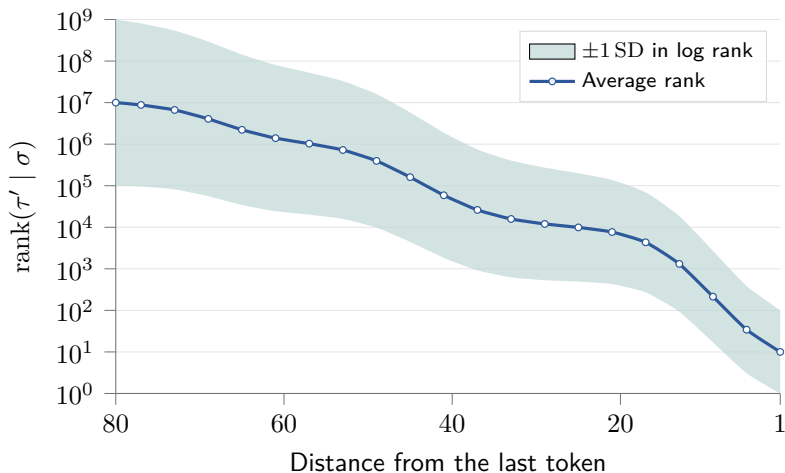
```
for (const auto& animal : animals)
    introduce(*animal, 2);
```

Normalized form admitted by G_Γ

```
for (const auto& {fresh_id} : animals)
    introduce(*{fresh_id}, 0);
```

Fresh-name handling (capture-avoiding). Only a binder introduced by the held-out statement, fresh relative to Γ , and all its bound uses may map to `{fresh_id}`. Keywords and contextual/API names stay exact; literals canonicalize to `0` or `""`.

Evaluation: Mock Results



At each prefix cutoff of a statement $\sigma\tau'$, σ is observed and τ' held out; the curve shows mean rank and variance. Hypothetical rank curve: simulated values only.

Future work

Scale up & validate

- ▶ Move from standalone files to full APIs and repository-scale corpora
- ▶ Test samples, report coverage, precision/recall, time, and memory

Increase expressivity

- ▶ Add fields, operators, conversions, variadics, and higher arities
- ▶ Target more languages; explore DCGs and richer type constraints

Smarter generation

- ▶ Materialize only productions reachable from the current prefix; quotient symmetric states
- ▶ Among CFG-admitted completions, use corpus or model scores to put likely continuations first

Performance optimizations

- ▶ **Lazy construction:** same bounded language, less memory?
- ▶ **Reranking:** same valid candidates, lower held-out rank?